



AI Usage — Transparency Report

Project: HRBrain – HR Management System

Date: 2026-05-04

Team: ZeroOne Studio (Mouadh Hamzaoui, Taieb Amine Belhadjali)

Note on transparency: *This document is written honestly. AI was used heavily throughout this project — not just for small tasks, but for large portions of infrastructure, testing, and documentation. We describe exactly what was generated, what failed, what we had to fix manually, and what decisions we made ourselves.*

1. AI Tools Used

Tool	What it is	How we used it
Kiro (Amazon)	Agentic AI IDE assistant	Primary tool — used for almost everything (see below)
Ollama + Qwen2.5:7b	Local LLM running on our cluster	Powers the in-app recommendation and NLP features
GitHub Copilot	VS Code inline code completion	Minor use — occasional boilerplate suggestions

2. The Main Agent: Kiro

Kiro is an AI development assistant built into our IDE. It works as an **autonomous agent**: we give it a task in natural language, and it reads files, writes code, runs terminal commands, and verifies results — all on its own.

We used Kiro in **Autopilot mode**, meaning it could make changes directly without asking for confirmation at every step.

The underlying LLM is **Claude** (Anthropic), accessed through Amazon's infrastructure.

This entire conversation — from Kubernetes setup to this document — was conducted with Kiro as the AI agent.

3. What AI Actually Did (Honest Breakdown)

3.1 Kubernetes & Infrastructure

Kiro generated:

- All Kubernetes YAML files (`k8s/` folder — 15+ files)
- Jenkins deployment on Kubernetes with NFS storage
- 4 CI/CD Jenkinsfiles (CI Backend, CI Frontend, CD Backend, CD Frontend)
- Prometheus + Grafana monitoring stack (configmaps, deployments, alert rules)
- Node Exporter DaemonSet
- kube-state-metrics deployment

What went wrong / required human intervention:

- The backend pods went **Pending** due to anti-affinity rules conflicting with only 2 available workers. Kiro tried multiple fixes (changing image tags, scaling ReplicaSets) before we decided to abandon application-level metrics entirely.
- Worker-1 ran out of disk space. Kiro provided the `growpart` + `resize2fs` commands, but we had to physically run them on the machines.
- Jenkins was initially configured for the wrong branch (`feature/k8s-jenkins-cicd` instead of `main`). We caught this and corrected it.

Human decision: We decided worker-3 (192.168.1.13) would be dedicated to Ollama only. Kiro respected this constraint once we stated it clearly.

3.2 Backend — NestJS

Kiro generated:

- 183 unit tests across 14 files (from ~40% to 87.87% coverage)
- Fixed code smells: moved hardcoded JWT secret to `ConfigService` , fixed regex escapes, removed unused imports
- Added Gzip compression middleware (`compression` package in `main.ts`)
- Added HTTP `Cache-Control` headers on API endpoints
- Created the full metrics module (`MetricsService` , `MetricsController` , `MetricsMiddleware`)

What went wrong:

- The metrics module was built and deployed, but the pods couldn't schedule due to anti-affinity. We rolled back to the previous working state. The metrics code still exists in the codebase but is not active in production.

- Some generated tests were too shallow at first (just checking that a function exists). We asked Kiro to rewrite them to test actual business logic.
-

3.3 Frontend — React + Vite

Kiro generated:

- 52 accessibility-specific tests (for `useFocusTrap`, `useFocusAnnouncer`, `CursorContext`, `ReadingMaskContext`, `useAppTranslation`)
- Updated `vite.config.ts` with code splitting (`manualChunks`) and image optimization (`ViteImageOptimizer`)
- `serve.json` with per-file-type cache headers
- SonarQube configuration files

What went wrong:

- Initial test coverage was 55.3%. Kiro's first attempt at new tests brought it to ~75%. We had to ask again with more specific instructions to reach 93.23%.
 - The `--testPathPattern` flag doesn't exist in Vitest (it's a Jest flag). Kiro used it initially and got an error, then corrected to the right Vitest syntax.
-

3.4 Documentation

Kiro generated:

- `WCAG-ACCESSIBILITY-AUDIT.md` — full accessibility audit report
- `PERFORMANCE-REPORT.md` — web performance report
- `MONITORING-SUMMARY.md` — monitoring stack documentation
- This file (`AI-USAGE.md`)

What we changed:

- The first versions of the WCAG and Performance reports were too long and technical (1000+ lines). We asked Kiro to rewrite them in simpler language suitable for a 10-minute academic presentation. This is the current version.
 - The first attempt to write `PERFORMANCE-REPORT.md` failed 6 times due to a tool error (`fsWrite` receiving null). We had to use a PowerShell `Out-File` workaround instead.
-

4. Prompts We Used

These are real prompts from our conversation with Kiro:

Setting up the cluster:

"Set up a complete Kubernetes cluster. Master is 192.168.1.10, worker-1 is 192.168.1.11 (Ahmed's laptop), worker-2 is 192.168.1.12 (Taieb's laptop), worker-3 is 192.168.1.13 — this one runs Ollama only, don't deploy anything else there."

Fixing disk pressure:

"je augmente l'espace disk de worker 1" (I'm increasing the disk space of worker 1)

Increasing test coverage:

"maintenant on veut augmenter le coverage de front entre 80%-90% et corriger les code smells dans le back et front" (now we want to increase frontend coverage to 80-90% and fix code smells in back and front)

Rolling back metrics:

"attend laisser le prometheus sans le hrbrain-backend, et retourner tous comme il est, on va passer pour une chose plus importante" (wait, leave Prometheus without hrbrain-backend, restore everything as it was, we'll move on to something more important)

Starting the accessibility audit:

"avant ca on concentrer maintenant dans la partie 'Accessibility Audit (WCAG)'" (before that let's focus now on the Accessibility Audit part)

Simplifying the reports:

"je veux que les 2 rapport (WCAG et performance) sont simple pour en peut comprendre et expliquer durant la validation" (I want both reports to be simple enough to understand and explain during the validation)

5. Where We Critically Evaluated AI Output

We did not accept everything Kiro produced. Here are cases where we pushed back or overrode it:

Situation	What Kiro did	What we decided
Anti-affinity + 2 workers conflict	Tried to fix scheduling by changing image tags and scaling ReplicaSets	We decided to abandon application metrics entirely rather than weaken the anti-affinity rules
Metrics pods Pending	Suggested changing <code>required</code> to <code>preferred</code> anti-affinity	We rejected this — HA was more important than metrics
Test quality	First tests were too shallow	We asked for rewrites with real logic testing
Report length	Generated 1000-line reports	We asked for simpler versions for the presentation
Wrong branch in Jenkins	Configured for <code>feature/k8s-jenkins-cicd</code>	We caught it and corrected to <code>main</code>
<code>fsWrite</code> tool failure	Failed 6 times silently	We used a PowerShell workaround

6. What We Did Without AI

Task	Done by
Physical machine setup (network, SSH, kubeadm init)	Team manually
Deciding the overall architecture (K8s, NFS, anti-affinity)	Team
Running <code>growpart</code> / <code>resize2fs</code> on physical machines	Team manually
Choosing worker-3 as Ollama-only node	Team decision
Reviewing every generated file before committing	Team
Deciding to abandon application metrics	Team decision
Writing the actual application features (HR logic, UI design)	Team (before this project phase)

7. Honest Assessment

What AI did well:

- Generating repetitive but correct YAML, test files, and configuration
- Remembering context across a long conversation (82+ messages)
- Recovering from errors and trying alternative approaches
- Explaining what it was doing and why

What AI did poorly:

- Sometimes generated code that looked correct but had subtle issues (wrong CLI flags, wrong branch names)
- Occasionally tried the same failing approach twice before reconsidering
- The `fsWrite` tool had a bug that caused 6 consecutive failures with no clear error message
- Some generated tests were superficial on the first attempt

Our role as developers:

- We provided the constraints (network topology, node roles, coverage targets)
- We validated every output by running tests and checking pod status
- We made the architectural decisions that AI cannot make (what to prioritize, what to sacrifice)
- We caught errors that AI missed (wrong branch, wrong flag names)

8. Summary

Primary AI Agent: Kiro (Amazon) – Claude-based LLM, Autopilot mode
In-App LLM: Qwen2.5:7b via Ollama (self-hosted on Kubernetes)
Code Completion: GitHub Copilot (minor use)

Approximate AI contribution by area:

Infrastructure:	~85% generated by AI, ~15% human correction
Backend tests:	~90% generated by AI, ~10% human review/rewrite
Frontend tests:	~80% generated by AI, ~20% human iteration
Documentation:	~70% generated by AI, ~30% human direction/simplification
Architecture:	~10% AI suggestion, ~90% human decision

All AI output was reviewed, tested, and committed by the team.
No code was committed without running tests first.